KubeSentinel AI

Observe clearly.
Recover with evidence.

Technical report
Architecture, implementation, experiments and release gates

An evidence-backed engineering portfolio. Synthetic evaluations, isolated lab results and illustrative cost assumptions are labeled throughout. No production deployment or published v1.0.0 release is claimed.

Contents and reading guide

01 Abstract and introduction

02 Problem statement, objectives and requirements

03 Architecture and technology justification

04 Component implementation and API contracts

05 Kubernetes operator design

06 ML methodology and evaluation

07 Incident intelligence and retrieval

08 Security and threat model

09 AWS deployment and GitOps ownership

10 Observability and service objectives

11 Test strategy and reproducibility

12 Experiments and recovery evidence

13 Load testing, scalability and reliability

14 Cost and operational considerations

15 Limitations and future work

Appendices: reproducibility commands, evidence inventory, planned experiments, references and release checklist.

Repository: https://github.com/manikantbindass/KubeSentinel-AI

01 Abstract and introduction

KubeSentinel AI is a Kubernetes operations platform that connects telemetry, incident analysis and policy-governed recovery. The repository contains a runnable Next.js/FastAPI foundation, durable PostgreSQL state, Redis coordination, local Kubernetes packaging, advisory anomaly detection and an independently privileged Go recovery controller. This report describes what the code and recorded experiments establish, and where further validation is required.

The central design principle separates evidence from authority. A detector may identify an unusual condition, and an assistant may explain it, but neither can execute a cluster change. Typed policies, current workload state and explicit approvals govern action execution. Verification then checks whether the workload and sampled service indicators meet the configured postconditions.

The evidence spans local unit tests, Linux kind CI, explicitly synthetic ML experiments and a small Windows loopback benchmark. It does not establish production accuracy, sustained availability or performance gains over native Kubernetes. AWS infrastructure exists as source only, and version 1.0.0 remains blocked by unresolved security gates.

Repository references

docs/architecture/system-overview.md

docs/releases/v1.0.0-candidate.md

02 Problem statement, objectives and requirements

Kubernetes restores desired replica counts and restarts failed containers, but operational diagnosis often requires joining metrics, logs, events and deployment changes. Alert storms and fragmented context increase review effort. Additional automation introduces its own risks: repeated restarts, incorrect rollbacks, availability loss and changes outside an authorized namespace.

The project objectives are to preserve evidence provenance, expose actionable incident context, constrain remediation through explicit policy, and record a durable explanation of each decision and outcome. A useful local development environment must run without AWS and must distinguish missing telemetry from synthetic fixtures.

Functional requirements include versioned API contracts, persistent incidents, bounded telemetry queries, deterministic feature windows, explainable anomaly scores, namespace-scoped approvals, recovery audit trails and responsive operations views. Nonfunctional requirements include least privilege, reproducible dependency resolution, explicit failure behavior, resource limits, accessible UI states and reviewable delivery artifacts.

Acceptance is environment-specific. Unit tests establish contract behavior; isolated kind tests establish selected cluster flows; a cloud demonstration requires separate credentials, cost approval and deployment evidence. No single green health endpoint proves all of these requirements.

Repository references

docs/roadmap.md

docs/security/threat-model.md

03 Architecture and technology justification

The context view places human operators, approvers, the identity provider, Kubernetes, GitHub and optional AWS services around the platform. The container view separates the web boundary, API, collectors, stores, advisory detector and privileged operator. Component views split authorization, query services, incident lifecycle, policy proposals and audit responsibilities inside the API.

Next.js and TypeScript support a typed operations experience and server-side API mediation. FastAPI and Pydantic align API validation with the Python ML toolchain. SQLAlchemy and Alembic make schema evolution explicit. PostgreSQL owns durable operational state; Redis accelerates bounded transient coordination. A PostgreSQL-backed queue is a deliberate alternative to Redis Streams, avoiding a second authoritative queue.

Go is used for controller reconciliation and a separate execution boundary. Prometheus handles time-series metrics, Loki handles logs, and OpenTelemetry normalizes application telemetry. Terraform owns AWS prerequisites, while Helm and Argo CD describe application delivery. Local kind remains the primary cluster validation environment.

Reasonable alternatives are React/Vite for a simpler SPA, a Go API for measured CPU pressure, Flux for GitOps, managed monitoring for lower operational burden and a PostgreSQL-only transient path at small scale. Adding microservices, a graph database or a deep model needs evidence of a concrete isolation, query or data-volume requirement.

Repository references

docs/diagrams/c4-context.mmd

docs/diagrams/c4-container.mmd

docs/diagrams/c4-component.mmd

docs/architecture/system-overview.md

04 Component implementation and API contracts

The backend exposes /api/v1 health, readiness and version contracts alongside operations endpoints. Health checks process liveness; readiness evaluates dependencies. Typed settings cover origins, dependency locations and feature switches. Structured JSON logs carry correlation and trace context; centralized exception handling avoids leaking internal stack details through normal API responses.

PostgreSQL records survive process restarts. Alembic migrations run before application startup. Redis supports cache, rate-limit and wake-up behavior; durable queue work remains in SQL. Connection pools, admission limits and bounded retries reduce unbounded accumulation. Exactly-once delivery is not assumed: idempotency and conflict checks matter at every mutation boundary.

The dashboard consumes typed query contracts through a server-side proxy. Twelve sections expose inventory, workloads, metrics, anomalies, incidents, recovery, policies, insights, audit, cost scenarios and preferences. SSE reconnects and polling provide change updates. Missing upstreams produce offline, error or empty states rather than switching silently to demo data.

Real mode is the deployed default. Synthetic fixtures require explicit enablement and a separate data partition. Local development authentication is restricted to trusted development setup. The public greeting/status surface is a smaller vertical slice and does not imply authorization for operational mutations.

Repository references

docs/development-guide.md

docs/runbooks/operations-dashboard.md

apps/api/kubesentinel_api

05 Kubernetes operator design

RemediationPolicy resources describe namespace and workload scope, incident types, allowed actions, severity thresholds, dry-run, approvals, cooldowns, budgets, healthy-replica constraints, maintenance windows, escalation and verification. RemediationAction records bind an incident and policy version to an exact target and action. The operator never interprets arbitrary shell commands or LLM instructions.

The reconciliation state machine observes a condition, qualifies policy, records a dry-run or rejection when appropriate, waits for signed approval when required, persists intent before mutation, applies a bounded change, and verifies postconditions. Supported action paths are controlled rollout restart, bounded scale and rollback to a verified previous revision. Critical namespaces and arbitrary pod deletion are excluded.

Leader election, durable intents, idempotency keys and workload conflict checks reduce duplicate execution. A target identity or template change, stale incident, expired approval, action-budget exhaustion or unavailable control plane can deny execution. Approval signatures bind the exact plan; editing the plan requires new authorization.

Verification combines rollout/readiness/replica observations with bounded service samples. Success is recorded only when configured checks pass. Failed verification can trigger a separately allowed undo or escalation. Sampled service checks are not proof of a long-term SLO, and rollback may itself fail. The controller is intentionally scoped rather than a general-purpose self-healing authority.

Repository references

docs/architecture/recovery-operator.md

docs/runbooks/recovery-operator.md

operator/crds

06 ML methodology and evaluation

The training pipeline records telemetry provenance and timestamps, creates deterministic feature windows, handles missing and duplicate observations, and computes rolling statistics and lag features. Features cover resource pressure, restart frequency, latency/error conditions, pending duration and replica availability. Time-based splits keep future windows out of training and threshold tuning.

The available detectors are a rule baseline, robust statistics and Isolation Forest. Metadata records the dataset hash, model identifier, runtime, threshold and score semantics. Inference scores are explicitly uncalibrated. Deterministic rules remain the default and fallback; no LSTM, autoencoder or calibrated probability is claimed.

The committed held-out evaluation contains 177 synthetic windows, including 20 positive windows from two fault incidents. Rules achieved precision/recall 1.00/1.00 on this fixture; robust statistics achieved 0.875/0.35; Isolation Forest achieved 0.667/0.90. These results reflect a tiny generated distribution and cannot be generalized to production clusters. The rule generator alignment is an important source of optimism.

Detection delay is first positive window end minus fault start and excludes missed incidents. Rules detected two incidents with mean delay 60 seconds; robust statistics detected one with 240 seconds and missed one; Isolation Forest detected two with 120 seconds. Warm serial inference latency is a separate measurement from end-to-end detection. Real-label validation, calibration and longer drift studies remain outstanding.

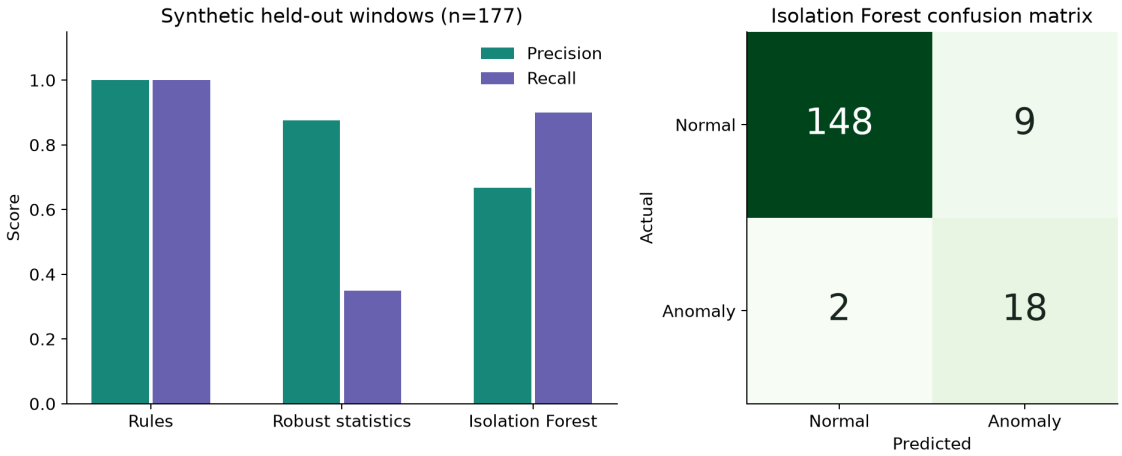


Figure source: docs/reports/phase-05/evaluation.json. Scope and limitations are stated above.

Repository references

docs/reports/phase-05/evaluation.json

ml/models/model-card.md

ml/models/synthetic-v1-metadata.json

07 Incident intelligence and retrieval

The AIOps layer groups related incidents using bounded time and namespace context, workload relationships and deployment-change evidence. It ranks hypotheses and historical similarity without asserting a confirmed root cause. Runbook retrieval uses a curated internal catalog and produces evidence links, uncertainty and read-only troubleshooting suggestions.

The Phase 10 evaluation contains eight explicitly synthetic hand-authored incidents with 12 expected directed links. All expected links and eight retrieval targets matched, with no extra links in that fixture. These are contract tests, not production correlation accuracy or a hallucination-rate estimate. The result file records zero remote requests.

Untrusted logs are treated as data rather than instructions. Secret/PII filtering, bounded context, rate limits and deterministic tool boundaries reduce exposure. The current no-LLM path avoids external provider egress. Any future provider needs privacy review, budget limits, outage handling and independent evaluation; it must not gain approval or execution privileges.

Repository references

docs/reports/phase-10/results.json

docs/architecture/0010-aiops-intelligence.md

08 Security and threat model

Primary threats include stolen user sessions, forged or replayed approvals, broad Kubernetes permissions, poisoned telemetry, prompt injection, secret leakage, supply-chain compromise and unbounded resource use. The browser is outside the execution trust boundary. OIDC-compatible roles and namespace scopes are enforced server-side, and the controller independently checks policy and approval evidence.

Approval signing keys belong in protected runtime secret storage, never Git. Exact origin configuration, secure session cookies and token validation protect the web boundary. Deployed identity uses issuer/audience/signature/expiry checks. Opaque OIDC access tokens and automatic refresh are not supported; live provider interoperability remains a certification gate.

Images use non-root identities and Kubernetes manifests apply scoped RBAC, security contexts, probes and network controls. Example admission policies and secret-manager integration require environment-specific rollout. OIDC federation replaces long-lived AWS keys for authorized delivery. Terraform state, plan files and credentials are ignored and must remain outside tracked source.

Security controls are not a claim of vulnerability-free software. Current image scans contain HIGH/CRITICAL findings and block release. Static scan failures and any dated exceptions must be reviewed under the repository policy. The release must not bypass a failing gate merely because a finding is upstream or currently unfixed.

Repository references

docs/security/threat-model.md

SECURITY.md

docs/runbooks/secure-delivery.md

09 AWS deployment and GitOps ownership

Terraform source defines reusable networking, EKS, IAM, ECR and observability modules, dev and demonstration configurations, validated inputs and tagging. The intended topology uses public/private subnet separation, private worker networking, restricted cluster access, managed nodes, encryption and scoped access entries. Optional DNS integration does not silently create a load balancer.

Remote state bootstrap is a separate lifecycle with encryption, versioning, access controls and locking instructions. AWS credentials are not embedded. GitHub OIDC trust is constrained to the repository and approved ref/environment. Workload permissions use scoped Pod Identity where supported. No live plan, apply or destroy is claimed in this report.

Terraform owns cloud prerequisites; Argo CD owns reviewed application configuration and immutable image digests. Actions build, test, generate SBOMs, scan and prepare approved artifact publication. Environment gates separate staging and production. Delivery starts disabled, and an operator must authorize costly or sensitive steps.

Rollback requires selecting a previously verified digest/configuration and reviewing the environment change. It is not a substitute for a database migration rollback plan. Teardown must inventory retained images, logs, state versions, volumes and keys. No AWS deployment or release publication occurs as part of portfolio generation.

Repository references

infrastructure/terraform/README.md

docs/runbooks/aws-eks.md

docs/runbooks/secure-delivery.md

10 Observability and service objectives

Prometheus and kube-state-metrics expose workload and node state; OpenTelemetry instruments the application and routes telemetry; Loki stores logs; Grafana presents operational views; Alertmanager routes bounded alert signals into incident normalization. Metrics, logs and events remain different evidence types with timestamps and provenance.

Useful signals include CPU/memory utilization, restarts, OOMKilled events, readiness, pending time, node pressure, HTTP request/error/latency metrics, deployment availability and remediation outcomes. Recording and alert rules reduce repeated query work and define severity. Labels must avoid unbounded IDs and user-controlled high-cardinality values.

The API uses a query abstraction with fixed scoped queries and explicit timeout/outage behavior. Monitoring failures do not fabricate healthy measurements or substitute synthetic data. Retention, log redaction and storage limits need operator sizing; the local stack is not an HA monitoring claim.

SLIs and error budgets are defined as operational goals, not achieved uptime. CPU/memory overhead, real incident frequency, queue lag and HPA response still need machine-readable repeated measurements. Actual dashboard screenshots are retained with their provenance; demo captures cannot establish real infrastructure performance.

Repository references

docs/runbooks/observability.md

docs/reports/phase-04-validation.md

11 Test strategy and reproducibility

The quality strategy combines backend lint/type checks and pytest, frontend lint/type/component tests and production build, migration tests against disposable PostgreSQL, Go controller tests, Helm/schema checks, browser flows and Linux kind integration. Locked npm/uv dependencies and pinned CI actions make inputs reviewable, although container base-image vulnerabilities remain an active release concern.

Local commands and exact results are captured in the Phase 11 validation record. Tests that require a database, sockets, Docker or a real cluster must not be reported as passed when unavailable. Historical CI evidence is identified by source commit and workflow URL; it cannot override a newly failing check.

Destructive fault injection requires explicit authorization and isolated disposable environments. The lab records context, namespace, policy, actor, action, result and timestamps. New chaos scenarios should run one at a time with cleanup and availability bounds. Broad node/network disruption is still pending.

Documentation validation checks local links, required files, license preservation, Markdown formatting and Mermaid parsing. Presentation validation checks package integrity, editability and layout, followed by rendering review. PDF text extraction complements visual inspection but cannot by itself prove absence of clipping.

Repository references

docs/reports/phase-11-validation.md

docs/reports/phase-06/summary.json

docs/reports/phase-07/browser-results.json

12 Experiments and recovery evidence

The isolated Phase 06 kind run records eight RemediationAction objects across approved, rejected and dry-run paths. These are action records, not eight independent comparable recovery trials. Safety rejection is an intended outcome and must not be counted as a failed recovery or used to inflate success rates.

For the approved scale example, observation was recorded at 02:48:56 UTC, application at 02:49:06 and success at 02:49:42 on 2026-11-03. The resulting 10-second observation-to-apply interval includes human approval and reconciliation polling; the 36-second verification interval is one lab observation. Timestamp resolution is one second.

There is no paired repeated native-Kubernetes baseline. Consequently this report makes no percentage improvement, average recovery SLA or false-remediation-rate claim. Future experiments must specify workload, cluster size, fault onset, repetitions, baseline treatment, confidence intervals and raw results.

The generated outcome and incident-type/severity charts describe only the stored lab action sample. They are not incident-frequency estimates for a production cluster. Detection delay comes from the separately labeled synthetic ML evaluation; combining those samples into an end-to-end recovery metric would be invalid.

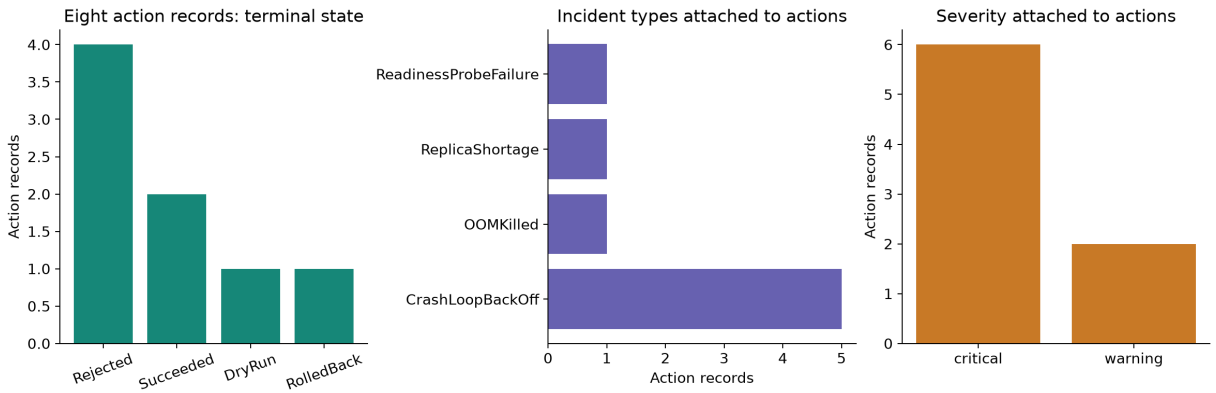


Figure source: docs/reports/phase-06/all-actions.json. Scope and limitations are stated above.

Repository references

docs/reports/phase-06/all-actions.json

docs/reports/phase-06/approved-scale.json

13 Load testing, scalability and reliability

The available loopback benchmark sends 900 total health requests on Windows: 100 requests per trial, three repetitions each at concurrency 1, 4 and 8. It exercises HTTP health only, with database_exercised=false and cluster_exercised=false. The observed per-trial p95 range is 16.6 to 31.0 ms and the sample contains no HTTP errors.

The charts retain individual trial values. Short trials are sensitive to scheduler, connection setup and warm-up effects; trial percentiles cannot simply be averaged into a pooled request percentile. Higher loopback throughput with concurrency is not evidence of horizontal scaling or PostgreSQL capacity.

Implemented controls include bounded queue admission, retry limits, connection-pool bounds, an ordered queue index, resource limits, stabilized API HPA configuration and graceful shutdown paths. The PostgreSQL queue is authoritative, while Redis accelerates notifications. Broader multi-worker behavior, queue backlog, distributed command claiming and failure recovery need dedicated measurement.

Planned experiments must capture CPU/memory, queue age, worker concurrency, database latency, p50/p95/p99 API latency, error rate and replica transitions under fixed workloads. HPA acceptance needs metrics availability and a sufficiently long run; node autoscaling needs a separately cost-approved environment. Backup/restore instructions are operational plans until recovery is actually rehearsed.

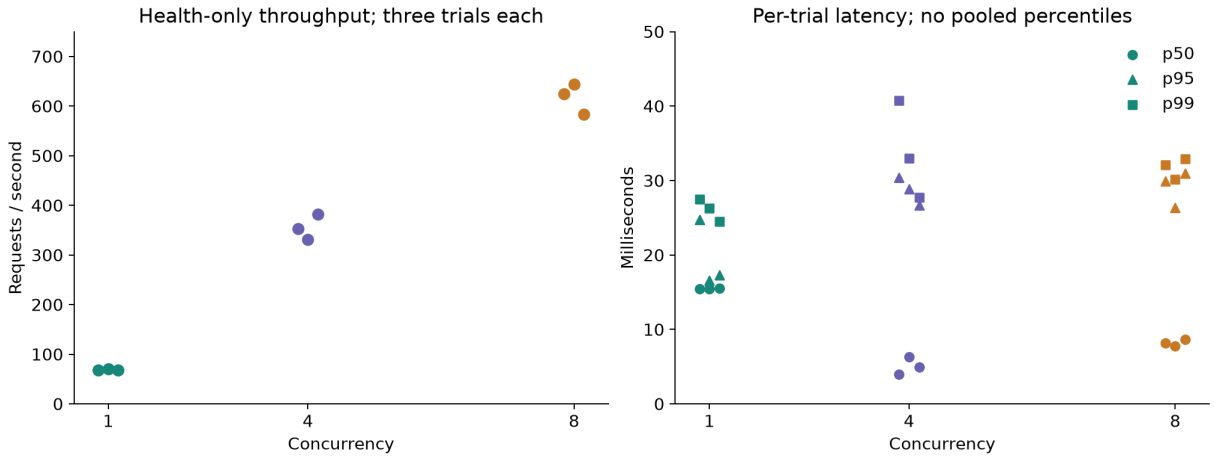


Figure source: docs/benchmarks/phase-09/loopback/summary.json. Scope and limitations are stated above.

Repository references

docs/benchmarks/phase-09/loopback/summary.json

docs/runbooks/resilience.md

14 Cost and operational considerations

The planning model assumes 730 hours/month, EKS at $0.10/cluster-hour, nodes at $0.10/node-hour, NAT at $0.05/gateway-hour plus $0.05/GB, gp3 at $0.10/GiB-month, 20 GB NAT processing and a $25 allowance for logs, ECR, state, KMS and requests. These are rounded illustrative inputs, not current regional AWS prices.

The dev example totals $211.50/month for one node and one NAT; the demonstration example totals $324.00 for two nodes and two NATs. Seven interface services in two AZs at an assumed $0.01/AZ-hour add $102.20 before processing. These subtotals exclude managed database/cache, load balancers, tax/support, transfer and other charges, and therefore underestimate a complete application environment.

Single NAT reduces baseline cost but adds zonal dependence and cross-AZ exposure. Private endpoints can cost more at low traffic and do not replace all internet egress. Logs, retained state and images continue charging after an application stops. EKS control-plane charges persist with zero nodes.

Before a cloud run, obtain dated account/region pricing, approve a ceiling and TTL, and define teardown evidence. Budget alerts are notifications, not spending stops. Cloud cost savings are not measured by this repository, and no AWS resources have been created for this report.

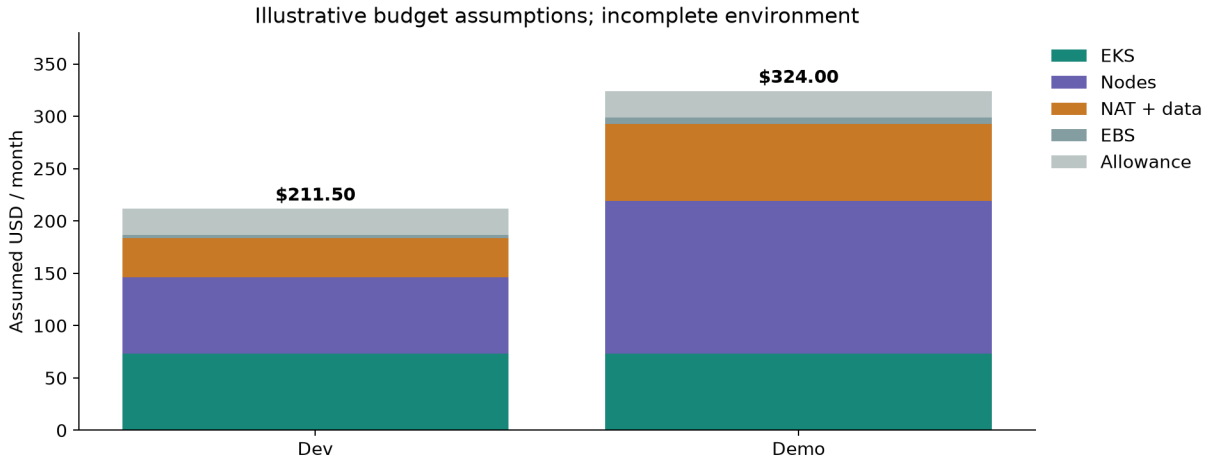


Figure source: docs/costs/eks-estimate.md. Scope and limitations are stated above.

Repository references

docs/costs/eks-estimate.md

docs/costs/aws-cost-considerations.md

15 Limitations and future work

The strongest current evidence is for scoped contracts and selected isolated integration paths. Synthetic ML and correlation data are small and hand-designed. Health-only loopback traffic does not establish full-stack throughput. Recovery observations are descriptive and lack repeated paired baselines. CPU/memory overhead, live scaling response and sustained SLO compliance remain unmeasured.

Production identity-provider certification, backup/restore rehearsal, managed database/cache deployment, high-availability monitoring, real GitOps promotion and cloud security enforcement are open gates. Queue and bridge topology require additional multi-replica testing. Optional LLM integration remains disabled and must not weaken the deterministic authorization boundary.

The immediate roadmap is to resolve vulnerability findings, repeat all CI gates, run repeated isolated recovery/load experiments, validate a live staging identity provider and rehearse digest promotion. A later cost-approved EKS demonstration must include teardown proof. Real-label model evaluation and read-only multi-cluster inventory follow evidence collection rather than preceding it.

Version 1.0.0 is a prepared release target, not a published release or a production readiness statement. The portfolio preserves original history and license, includes editable documentation sources and links every reported result to a machine-readable record. Reviewers should use the dated validation and release checklist to determine the current state.

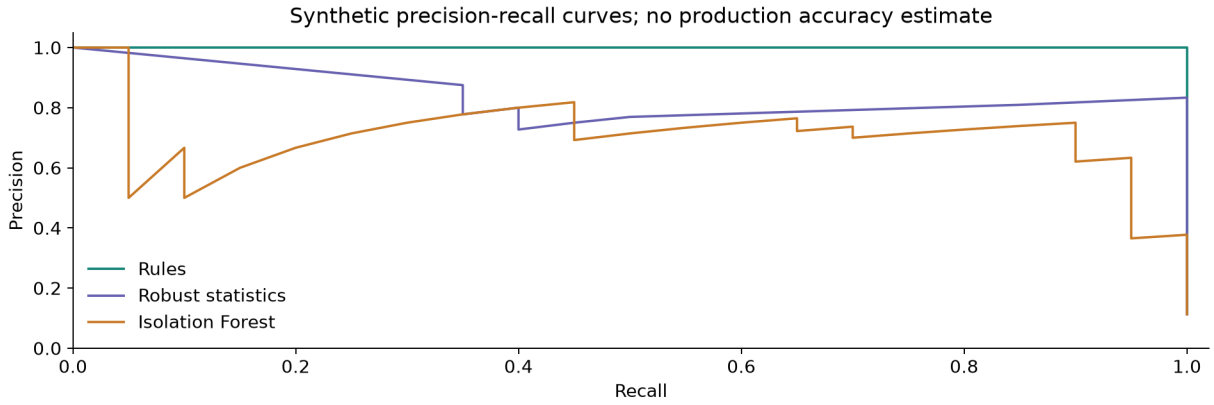
Repository references

docs/releases/v1.0.0-candidate.md

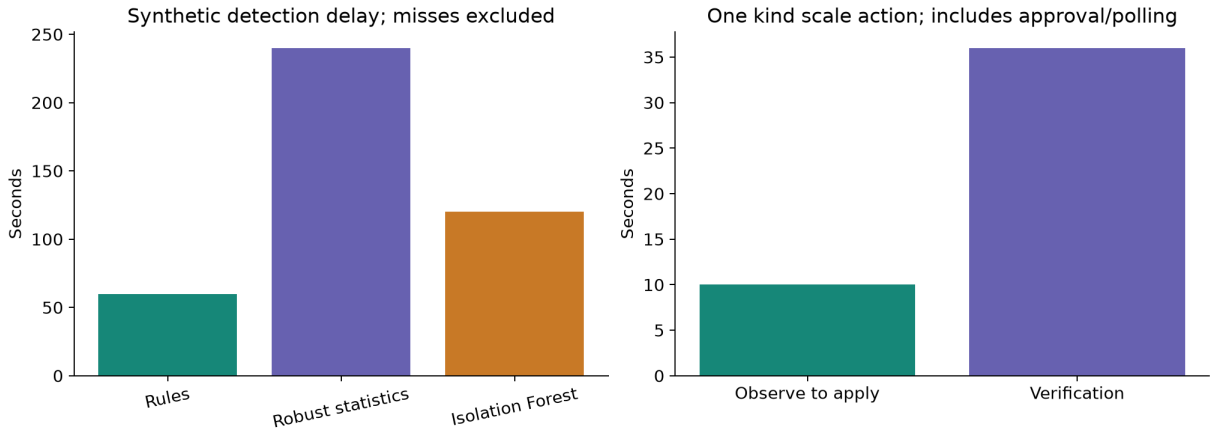
docs/roadmap.md

Supplementary evaluation plots

These plots use the same small synthetic test split and single isolated scale record described in the experiment chapters. Missed incidents are excluded from mean detection delay; robust statistics misses one of two incidents. Detection and recovery intervals are different samples and must not be added together.



Synthetic precision-recall curves from the committed per-detector CSVs.



Left: synthetic mean detection delay. Right: one kind scale action, including approval.

Sources: docs/reports/phase-05/*-precision-recall.csv; docs/reports/phase-05/evaluation.json; docs/reports/phase-06/approved-scale.json.

Appendix A Reproduction and validation

Locked application install

```
npm ci; uv sync --frozen --extra ml --extra ops
```

Frontend

```
npm run lint; npm run typecheck; npm test; npm run build
```

Backend

```
uv run ruff check .; uv run ruff format --check .; uv run mypy; uv run pytest -m 'not postgres'
```

Documentation

```
npm run lint:docs; npm run diagrams; uv run python scripts/validate_repository.py
```

Local Compose

```
docker compose up --build --wait --wait-timeout 180; python scripts/setup/smoke.py
```

Portfolio figures

```
uv run python docs/portfolio/build_figures.py
```

PowerShell and Linux/macOS setup details live in docs/development-guide.md. Operations authentication and demo seeding use docs/runbooks/operations-dashboard.md. Documentation runtimes and PowerPoint/PDF export commands are pinned in docs/portfolio/README.md.

A successful command on one source commit is historical evidence, not a guarantee for every later commit. See the dated Phase 11 validation matrix and CI snapshot for exact outcomes and unavailable dependencies.

Appendix B Evidence inventory

docs/reports/phase-05/evaluation.json

SHA-256: e0523f45a70eddc7e6e8d81a760ff4d51ef604e0413ec57d67d791698ee178cb

docs/reports/phase-06/all-actions.json

SHA-256: e8dacd412b2baf57baaceb9709c5f2028b35d64cbae665655c461af31425b3cf

docs/reports/phase-06/approved-scale.json

SHA-256: 651aef37235603272dffff5f7400776a7556d981bf687ea4295a7aab0e370596

docs/benchmarks/phase-09/loopback/summary.json

SHA-256: 2eb6cd2b7bcfe7788767e19481743fd52b991b52c4f7e6bf1978e95e5ba79ab3

docs/reports/phase-10/results.json

SHA-256: 5656b039238e3f470f98778a00c468a9a1e9ab4bf123c6c45aa3438b84ecd403

docs/costs/eks-estimate.md

SHA-256: 419559428f07b8484cb61765db538710b21d6842415465267fe9345bb426319b

docs/reports/phase-05/rules-precision-recall.csv

SHA-256: 1e196d6cfca095fe72605719426b7dbaba7ae0fe777c2dbdd9e21ecb6c6f4935

docs/reports/phase-05/robust_statistics-precision-recall.csv

SHA-256: de689bd1d0c57c8a8a23a4b3d7e4dce0422a77b3cb23bc96b1376a1ad7e0b6ba

docs/reports/phase-05/isolation_forest-precision-recall.csv

SHA-256: fdc44b041765b544f5b6c1a5a404ba5bc8436f83c50df8f303fbb8f1e91d8b51

Scope: Recorded synthetic evaluations and lab/loopback observations; costs are assumptions

The manifest hashes UTF-8 input text with Git LF line endings for portable provenance. Presentation charts contain editable literal data and embedded workbook snapshots. Editing a chart changes the presentation; regenerate from source to restore repository evidence.

Appendix C Planned experiments

Measurement	Required evidence before a result claim
CPU / memory overhead	Timestamped pod/node samples, requests/limits, scrape interval, baseline and repeated workload.
Scaling behavior	Replica transitions, sustained arrival rate, HPA inputs, queue age and node capacity over time.
Recovery comparison	Paired native/KubeSentinel trials, identical faults, fault onset and verification timestamps, failures retained.
Production ML accuracy	Consented real labeled data, temporal/incident split, tuning isolation, false positives and missed incidents.
Full-stack load	Authenticated ingestion/scoring/decision traffic, SQL/Redis latency, throughput and error distributions.

These are planned experiments, not illustrative benchmark values. No CPU, memory, scaling or recovery-improvement curves are fabricated to fill the evidence gaps.

Appendix D References and release

Project source and original Apache 2.0 license
https://github.com/manikantbindass/KubeSentinel-AI

Kubernetes controller model
https://kubernetes.io/docs/concepts/architecture/controller/

OpenTelemetry concepts
https://opentelemetry.io/docs/concepts/

Prometheus instrumentation practices
https://prometheus.io/docs/practices/instrumentation/

Terraform state
https://developer.hashicorp.com/terraform/language/state

AWS EKS pricing review
https://aws.amazon.com/eks/pricing/

AWS pricing calculator
https://calculator.aws/

The architecture choices in this report are project decisions. External references provide technology background; measured results come only from the repository records identified in each chapter and Appendix B.

Release checklist

Resolve blocking vulnerabilities; repeat the complete quality/security matrix; review artifacts and links; rehearse staging promotion; obtain maintainer authorization before merge or publication. No release tag, GitHub release or AWS deployment is created by this document build.

Acknowledgements: the Kubernetes, CNCF, Python, Go, React/Next.js and open-source tool communities. Original repository history and LICENSE are preserved. Third-party components retain their own licenses.